

Práctica 4 - SLAM con Filtro de Partículas (FASTSLAM) y Planeamiento de Trayectorias

Robótica Móvil

Prof. Dr. Ignacio Mas

29 de mayo de 2025

Fecha límite de entrega: 13/6/25, 23:59hs.

Modo de entrega: Enviar por el Aula Virtual del Campus el código (.m) comentado y los gráficos (.jpg ó .pdf) y/o animaciones.

1. Implementación de algoritmo FASTSLAM

En este ejercicio se implementará el algoritmo FASTSLAM basado en landmarks. Se asume que los landmarks son identificables por lo que el problema de asociación de datos está resuelto. El punto de partida es una estructura de código provista por la cátedra.

1.1. Notas preliminares

La estructura provista contiene los distintos componentes del algoritmo FASTLAM, para que el esfuerzo del desarrollo sea en los detalles de la implementación del algoritmo propiamente dicho. El archivo comprimido que se provee incluye las carpetas:

- **data** contiene la descripción del mundo y las lecturas del sensor
- **matlab** contiene la estructura del algoritmo FASTLAM con secciones para ser completadas
- **plots** guarda los gráficos generados como resultado

Para ejecutar el algoritmo, desde la carpeta *matlab* correr el archivo `fastslam.m`. Mientras que corre, los gráficos se van guardando en la carpeta *plots*. El algoritmo inicialmente está incompleto y no correrá correctamente. La estructura hace uso de la librería *librobotics*, escrita por Kai Arras (ASL-EPFL) para la visualización. Todas las funciones están definidas en la estructura de código.

Adicionalmente, se provee un archivo *notas_practica_fastslam.pdf* con detalles de la implementación del algoritmo FASTSLAM para usarse como guía.

Algunos consejos adicionales:

- Desactivar la visualización comentando la línea `plot_state(...)` en el script `fastslam.m` para acelerar la ejecución.
- Para probar el filtro sólo con algunos pasos, cambiar el tiempo final en el bucle *for* principal con, por ejemplo, `for t = 1:50`.

1.2. Paso de corrección FASTSLAM

Implementar el paso de corrección en el archivo `correction_step.m`. Asumir que el ruido de medición está caracterizado por la matriz diagonal cuadrada de 2×2 Q_t :

$$Q_t = \begin{pmatrix} 0,1 & 0 \\ 0 & 0,1 \end{pmatrix}$$

Nota: Con el comando *ffmpeg* (si está instalado en tu sistema operativo) se puede generar una animación de los gráficos de la carpeta *plots* de la siguiente manera:

```
ffmpeg -r 10 -i fastslam_%03d.png fastslam.mp4
```

2. Planeamiento de caminos

Los algoritmos de búsqueda en grafos como Dijkstra o A* pueden ser usados para planear caminos en grafos desde un lugar de inicio hasta un objetivo. Si las celdas de un mapa de grilla se representan como nodos conectados con sus celdas vecinas, estos algoritmos pueden aplicarse directamente para realizar planeamiento para robots. Para este ejercicio, consideramos las 8 celdas vecinas de una celda $\langle x, y \rangle$, que se definen como las celdas adyacentes a $\langle x, y \rangle$ horizontalmente, verticalmente y en diagonal.

El archivo incluido contiene una implementación de planeamiento en 2-D basado en grafos. El script `planning_framework.m` contiene la parte principal del algoritmo y es el que debe ejecutarse. Este archivo no necesita ser modificado, pero es aconsejable entender lo que hace. Los ejercicios de esta sección se realizan implementando las funciones vacías que acompañan al script principal.

2.1. Algoritmo de Dijkstra

El algoritmo de Dijkstra se usa para calcular caminos de costo mínimo dentro de un grafo. Durante la búsqueda, siempre se busca el nodo del grafo con el menor costo desde el punto de inicio y se agregan los nodos vecinos al grafo de búsqueda.

1. Sea $M(x, y)$ un mapa de grilla de ocupación. Durante la búsqueda, las celdas se conectan con sus celdas vecinas para construir el grafo de búsqueda. Completar la función `neighbors` provista que define los vecinos de una celda. La función toma como entrada las coordenadas de una celda y el tamaño del mapa, y devuelve un vector de $n \times 2$ con las coordenadas de sus celdas vecinas, teniendo en cuenta los límites del mapa.
2. Implementar una función para los costos de un arco entre nodos que permita planear caminos de mínima longitud y libre de colisiones. Considerar la celda como un obstáculo si su probabilidad de ocupación supera cierto umbral. ¿Qué umbral se debería elegir? Implementar la función `edge_cost`.
3. Incluir información de ocupación en la función de costo que permita que el algoritmo elija celdas con baja probabilidad de ocupación sobre celdas con mayor probabilidad de ocupación.

2.2. Algoritmo A^*

El algoritmo A^* utiliza una heurística para realizar una búsqueda informada que resulta ser más eficiente que el algoritmo de Dijkstra.

1. ¿Qué propiedades debe tener dicha heurística para asegurar que A^* es óptimo?
2. Definir una heurística para planeamiento de robots móviles en 2-D. Completar la función `heuristic` provista. La función toma como entrada las coordenadas de una celda y del objetivo, y devuelve el costo estimado hasta el objetivo.
3. ¿Qué pasa si se aumenta la heurística usando h_2 , siendo h_2 un múltiplo de la heurística h definida en el punto anterior. Analizar el comportamiento con diferentes factores: $h_2 = \{1; 2; 5; 10\} * h$